

KineSync-GS: Verified Visual State Synchronization for Articulated Gaussian Twins

Anonymous Authors | Project Website: <https://anonymous-for-sub.github.io/KineSync-GS>

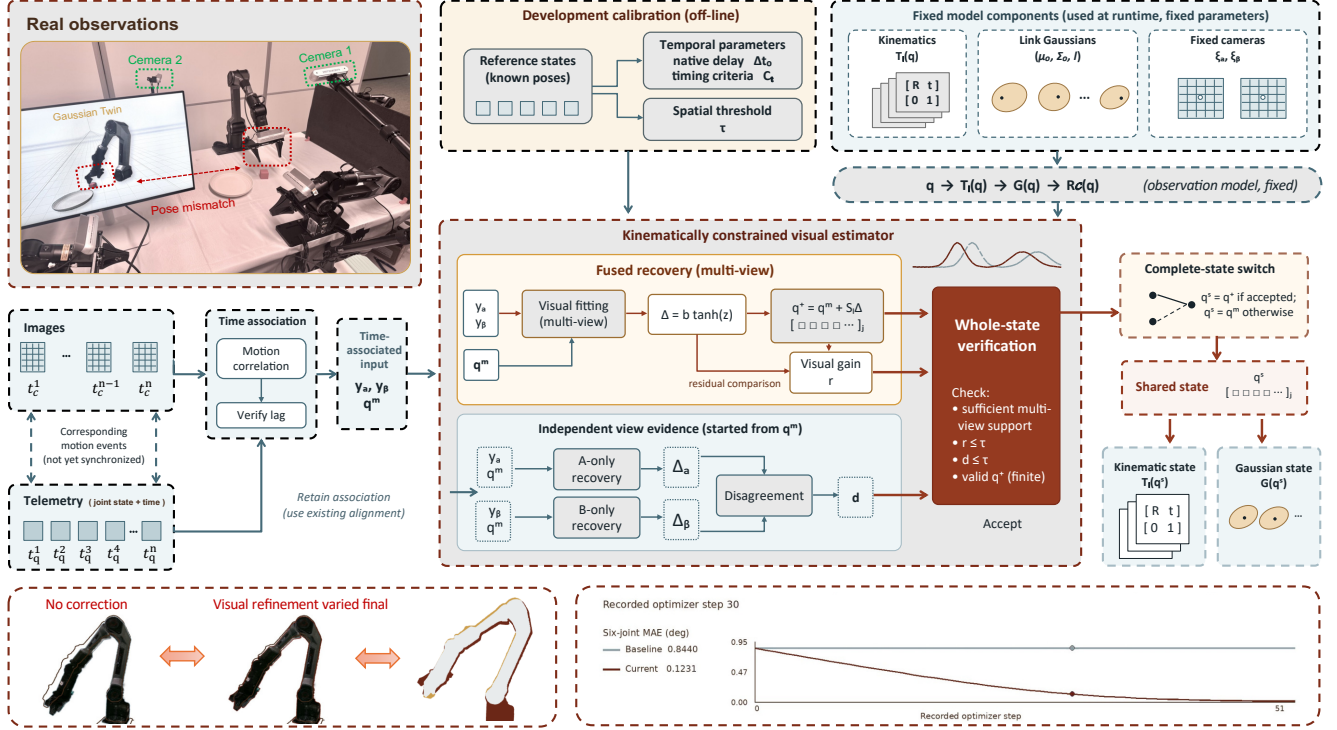


Fig. 1. KineSync-GS uses rendering or motion to propose a correction, cross-view or motion evidence to verify it, and a shared update/fallback interface to synchronize an articulated Gaussian twin.

Abstract—Gaussian robot twins can reproduce physical appearance with high fidelity, but their usefulness depends on synchronizing that appearance with the robot state that generated an observation. Direct visual fitting is insufficient: under inconsistent views, lower rendering error can correspond to a less accurate articulated state. We introduce KineSync-GS, a verified synchronization framework built around a single principle: an estimator proposes a correction, and evidence determines whether that correction should modify the twin. A kinematically bound Gaussian model produces bounded whole-state proposals, cross-view evidence verifies spatial updates, and motion evidence instantiates the same proposal-verification interface for camera-telemetry alignment. Only evidence-supported proposals are committed; otherwise the measured state is preserved. On twelve held-out Franka poses, KineSync-GS reduces joint error from 0.590° to 0.322° with no harmful whole-state update across 48 conditions. In online PiPER experiments, the complete pipeline reaches 3.07 mrad with 0.36% harmful updates, and supplying its synchronized state to an unchanged policy raises task-macro success from 58.33% to 71.67%. These results establish KineSync-GS as a reliable synchronization interface between real observations, articulated Gaussian twins, and downstream robot policies.

I. INTRODUCTION

Photorealistic simulation increasingly connects real observations with embodied learning. Gaussian representations capture visual detail beyond manually textured geometry, while articulated models provide interaction-ready physical structure [1], [2], [3]. Together they form a visual twin when the rendered robot matches the observation in state and time. Telemetry alone cannot guarantee correspondence: joint-zero error, calibration residuals, stale views, and asynchronous streams can displace the rendered embodiment. Images directly expose these discrepancies, while link-bound Gaussians make them differentiable with respect to joint state.

A better rendering fit, however, need not imply a better state. Under disagreeing views, optimization can lower image loss by moving the twin away from the robot. Figure 2 exposes this failure mode: with a stale observation, visual loss falls by 50.02% while joint error reaches 5.073° . State estimation and state-changing authority must therefore be

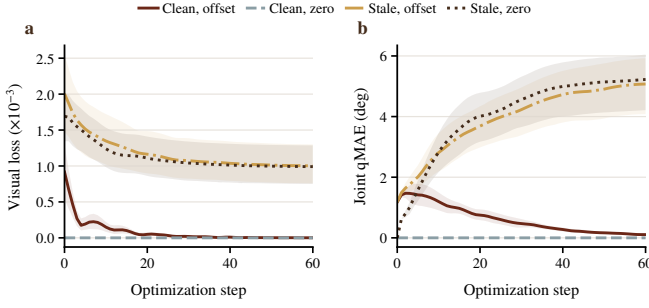


Fig. 2. Image fitting and state recovery can diverge. Curves aggregate all twelve held-out poses; bands show 95% pose-bootstrap intervals. Under stale views, optimization lowers visual loss while increasing state error.

separated.

We propose KineSync-GS, which separates correction proposals from state-changing authority. A kinematically bound Gaussian renderer first estimates a bounded, coordinated joint correction. View-specific estimates then provide cross-view evidence for committing the complete proposal or retaining the measurement. The same principle extends to time: image and joint motion propose a camera–telemetry offset, and calibrated motion evidence determines whether resampling is justified. Spatial and temporal synchronization therefore share one sequence: proposal, evidence, verified update or fallback. We evaluate this principle under controlled Franka view conflicts, real-world PiPER replay across two Gaussian backends, trajectory-level timing disturbances, and the complete online PiPER pipeline with a fixed manipulation policy. Across these settings, KineSync-GS turns visual evidence into a selective state-changing signal rather than merely a fitting objective.

Our contributions are: (1) A kinematically grounded Gaussian observation model that turns robot appearance into bounded whole-state proposals; (2) A unified proposal–evidence–update interface for spatial and temporal synchronization, combining cross-view whole-state verification with measurement-preserving fallback; and (3) Controlled, real-world, and online evaluations that connect synchronization accuracy to update reliability and downstream manipulation.

II. RELATED WORK

3D Gaussian Splatting (3DGS) provides explicit primitives and differentiable rendering [1]. RoboGSim and GS-Playground combine it with closed-loop simulation [4], [3], while RoboSplat edits reconstructed Gaussians across visual worlds [2]. RE³SIM, GSWorld, and reconstructed soft-body simulation connect Gaussian appearance with physics-backed policy learning [5], [6], [7]; SplatCtrl updates scenes from RGB-D streams for reactive control [8]. These systems provide the substrate; KineSync-GS controls how visual evidence updates its articulated state.

Articulated recovery spans distance-model tracking [9], camera-to-robot estimation [10], [11], joint camera/robot refinement [12], and rigid pose estimation [13]. Differentiable Robot Rendering optimizes joints through Gaussian residuals [14]; GaussTwin couples physical prediction with

visual correction [15]; SplArt and active-view methods recover articulation or improve observations [16], [17], [18]. Our known skeleton instead produces bounded telemetry corrections whose complete admission is decided by cross-view evidence.

Temporal calibration jointly estimates sensor offsets and transforms [19], or recovers delay by aligning motion curves [20]. KineSync-GS correlates visual and joint motion, removes a calibrated native delay, and verifies the correction before resampling. Spatial and temporal paths share an estimate–verify–retain interface, and the resulting synchronized state can be consumed directly by downstream policies such as OpenVLA-OFT [21].

III. VERIFIED VISUAL STATE SYNCHRONIZATION

A. State and Observation Model

Let $q^m \in \mathbb{R}^J$ denote the measured arm-joint state and $\mathcal{C}$ the available camera views. For camera c , y_c is the image observation and ξ_c its calibrated parameters. The system estimates q^+ and produces a synchronized state q^s . A reference state q^* is used to calibrate decisions on development data and to score test predictions; it is not an input to test-time verification.

Each Gaussian k is attached to a robot link $\ell(k)$ with local mean $\mu_{k,0}$ and covariance $\Sigma_{k,0}$. Forward kinematics gives the link rotation $R_{\ell(k)}(q)$ and translation $t_{\ell(k)}(q)$, yielding

$$\mu_k(q) = R_{\ell(k)}(q)\mu_{k,0} + t_{\ell(k)}(q), \quad (1)$$

$$\Sigma_k(q) = R_{\ell(k)}(q)\Sigma_{k,0}R_{\ell(k)}(q)^\top. \quad (2)$$

Rendering these transformed Gaussians defines an observation $\mathcal{R}_c(q; \mathcal{G}, \xi_c)$. A single articulated state therefore drives both the kinematic model and the Gaussian observation model.

Persistent calibration is separated from per-observation recovery. In the PiPER factorized model, training poses determine a shared joint-zero vector and one bounded SE(3) residual per camera, with the base world frame fixed. These quantities are held fixed when evaluating new state corrections. This factorization keeps persistent camera and joint calibration fixed while reserving per-observation optimization for the state correction itself.

B. Bounded Gaussian State Estimation

For selected joints I , let S_I insert a local correction into the full state. We optimize an unconstrained variable z through

$$\delta = b \tanh(z), \quad q^+ = q^m + S_I \delta, \quad (3)$$

where b specifies the correction bound. The visual objective averages observations over cameras:

$$\mathcal{L}(q) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \ell_c(\mathcal{R}_c(q), y_c) + \lambda_p \ell_{\text{prior}}(q, q^m). \quad (4)$$

The observation loss follows the available backend. PiPER supports soft-mask overlap, boundary alignment, and foreground RGB residuals with affine color fitting. The controlled Franka study uses mean RGB squared error over

the two views. Both feed the same bounded state interface, making the verification stage independent of the photometric backend.

A fused estimate uses all views. We also recover δ_A and δ_B from the same starting measurement using each view separately. These view-specific estimates provide cross-view evidence for validating the fused proposal. Their normalized disagreement is

$$d = \frac{\|\delta_A - \delta_B\|_2}{\max(\|\delta_A\|_2 + \|\delta_B\|_2, \epsilon)}. \quad (5)$$

When $\|\delta_A\|_2 + \|\delta_B\|_2 \leq \epsilon$, we set $d = 0$; we use $\epsilon = 10^{-8}$ throughout evidence computation. We measure visual improvement by

$$r = \frac{\mathcal{L}(q^m) - \mathcal{L}(q^+)}{\max(|\mathcal{L}(q^m)|, \epsilon)}. \quad (6)$$

For calibration, the evidence family additionally records gradient and correction-direction cosines,

$$g = \cos(P_I \nabla_q \ell_A(q^m), P_I \nabla_q \ell_B(q^m)), \quad (7)$$

$$a = \cos(\delta_A, \delta_B), \quad (8)$$

where P_I restricts both gradients to the same joint coordinates. Degenerate cosine denominators return zero. Evidence must be finite and use at least two views. Collecting these signals into $e^{\text{sp}} = (r, g, a, d)$ makes the state-changing rule explicit:

$$A_\tau(e^{\text{sp}}) = \mathbf{1}_{\text{finite}} \mathbf{1}_{|c| \geq 2} \mathbf{1}_{r \geq \tau_r} \mathbf{1}_{g \geq \tau_g} \mathbf{1}_{a \geq \tau_a} \mathbf{1}_{d \leq \tau_d}. \quad (9)$$

The fused renderer therefore owns proposal generation, while view-specific recoveries supply the evidence that controls state-changing authority.

C. Development-Calibrated Whole-State Verification

We calibrate when a visually improved estimate is allowed to become a state update. For a development example i , define

$$e_i(q) = \frac{1}{J} \sum_{j=1}^J |q_j - q_{i,j}^*| \quad (10)$$

and label an estimate helpful when $e_i(q_i^+) < e_i(q_i^m) - \epsilon_e$, or harmful when $e_i(q_i^+) > e_i(q_i^m) + \epsilon_e$. We use $\epsilon_e = 10^{-8}$ rad; neutral examples contribute to neither label.

We calibrate τ on development examples by lexicographically minimizing

$$(N_{\text{harm}}(\tau), -N_{\text{help}}(\tau), -M(\tau), \tau). \quad (11)$$

Here N_{harm} and N_{help} count accepted harmful and helpful estimates. M is the minimum signed threshold margin over helpful and harmful development examples, rewarding separation after the two count criteria. A deterministic threshold order breaks remaining ties. Thresholds are fixed after development; evaluation uses only the measurement, proposal, evidence, and fixed thresholds.

The final update is atomic at the state-vector level:

$$q^s = \begin{cases} q^+, & A_\tau(r, g, a, d) = 1, \\ q^m, & \text{otherwise.} \end{cases} \quad (12)$$

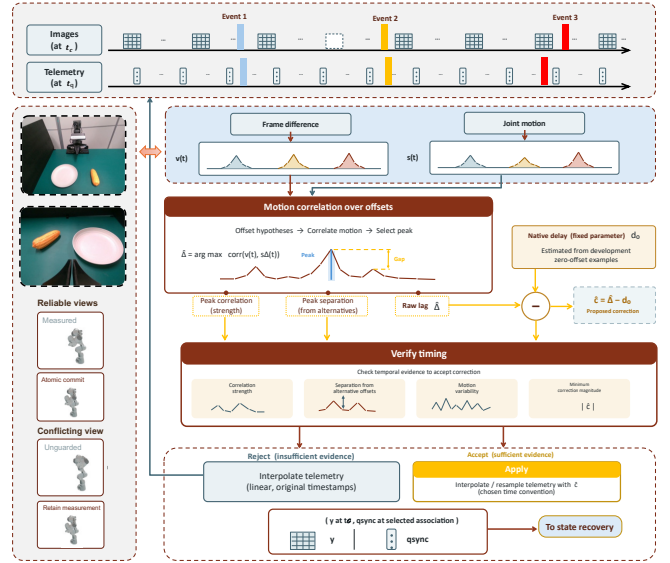


Fig. 3. Motion-based alignment of image and telemetry streams, followed by temporal verification and state resampling.

Thus an accepted correction preserves the coordinated estimate, and a rejected correction leaves the measurement unchanged.

In the Franka development split, calibration selects $\tau_r = 0$, $\tau_g = \tau_a = -1$, and $\tau_d = 0.42671$, yielding a simple operating rule based on finite multi-view support, nonnegative visual gain, and bounded cross-view disagreement.

a) *Why preserve the complete estimate?*: Articulated visual residuals couple multiple joints. Masking individual components after joint estimation changes that coordinated solution. For intuition, consider a linearized least-squares residual $u + J_r \delta$ and a stationary minimizer δ^* . If D is a diagonal component mask, then

$$\begin{aligned} \frac{1}{2} \|u + J_r D \delta^*\|_2^2 - \frac{1}{2} \|u + J_r \delta^*\|_2^2 \\ = \frac{1}{2} \|J_r (D - I) \delta^*\|_2^2 \geq 0. \end{aligned} \quad (13)$$

The cross term vanishes by stationarity. This local identity shows that coordinate selection can disrupt a useful coordinated correction, motivating verification and commitment of the fused estimate as a whole.

D. Real-World Visual Recovery

A second estimator instantiates the same estimate-verify interface using state-indexed Gaussian replay. Given a real PiPER head-camera image, we compare it with rendered states in a symmetric radius-four window around the delayed index in the selected frame sequence. Normalized RGB L_1 discrepancy selects the best matching state. A development-calibrated visual-gain threshold determines whether to use that state or retain the delayed input. A shared threshold of 0.00074274 is used for both Component-GS and Hybrid-GS.

This discrete estimator follows the same state contract as continuous whole-state recovery. Rendering proposes a state index, calibrated image evidence determines whether it may replace delayed telemetry, and rejection preserves the input measurement. The estimator and evidence differ

Algorithm 1 Cross-view whole-state synchronization

Require: Measured state q^m , views $\{y_c\}_{c \in \mathcal{C}}$, model $\mathcal{G}$, fixed thresholds τ

Ensure: Synchronized state q^s and verification record h

```

1: Validate calibrated views and initialize all estimators at  $q^m$ 
2:  $(q^+, \delta^+) \leftarrow \text{BoundedEstimate}(q^m, \{y_c\})$ 
3: for all  $c \in \mathcal{C}$  do
4:    $\delta_c \leftarrow \text{SingleViewEstimate}(q^m, y_c)$ 
5:    $\ell_c^m, \ell_c^+ \leftarrow \text{RenderLoss}(q^m, q^+, y_c)$ 
6: end for
7:  $(r, g, a, d) \leftarrow \text{CrossViewEvidence}(\{\ell_c^m, \ell_c^+, \delta_c\})$ 
8:  $v \leftarrow \text{Finite}(q^+, r, g, a, d) \wedge |\mathcal{C}| \geq 2$ 
9: if  $v \wedge A_r(r, g, a, d) = 1$  then
10:   $q^s \leftarrow q^+$ 
11:   $h \leftarrow (\text{ACCEPT}, r, g, a, d, \delta^+)$ 
12: else
13:   $q^s \leftarrow q^m$ 
14:   $h \leftarrow (\text{RETAIN}, r, g, a, d, \mathbf{0})$ 
15: end if
16: Record  $h$  with the fixed calibration identifier and input timestamps
17: return  $(q^s, h)$ 

```

across backends; the proposal–verification–update interface does not.

E. Motion-Based Temporal Alignment

Images and telemetry also provide complementary motion cues for time alignment. We form a visual motion trace $v(t)$ from grayscale head/wrist frame differences and a joint-motion trace $s_\Delta(t)$ under a candidate offset. The temporal estimate is

$$\hat{\Delta} = \arg \max_{\Delta \in \Omega} \text{corr}(v(t), s_\Delta(t)), \quad \hat{c} = \hat{\Delta} - d_0, \quad (14)$$

where $\Omega = \{-200, -180, \dots, 200\}$ ms and d_0 is the native delay estimated from development zero-offset examples. A temporal verifier tests peak correlation, separation from alternative offsets, motion variability, and minimum correction magnitude. Accepted estimates resample the state stream; otherwise the output remains the original linear interpolation.

More precisely, let $e^{\text{tm}} = (\rho_{\max}, \rho_{\text{gap}}, \sigma_s, |\hat{c}|)$ collect the four timing signals and let $B_\kappa(e^{\text{tm}})$ apply the fixed development thresholds κ . At camera time t_c , the published state is

$$q^s(t_c) = \begin{cases} \text{Interp}(q, t_c + \hat{c}), & B_\kappa(e^{\text{tm}}) = 1, \\ \text{Interp}(q, t_c), & \text{otherwise.} \end{cases} \quad (15)$$

This exposes the temporal proposal and its commit decision through the same measurement-preserving contract as Eq. (12).

Spatial and temporal estimators therefore implement the same sequence: construct a correction proposal, evaluate it with separate evidence, and either update the complete state or preserve the input. Each path uses evidence matched to its observations while exposing the same synchronized-state interface to rendering, simulation, and downstream policies.

IV. EXPERIMENTAL PROTOCOLS

A. Evaluation Settings

We organize the evaluation around four questions (Table I): whether whole-state verification resolves controlled

TABLE I

EVALUATION PROTOCOLS. PERTURBATIONS REMAIN WITHIN THEIR LISTED BOOTSTRAP UNIT.

Protocol	Split	Held-out evaluation	Unit
Franka spatial	6 / 12	4 conditions $\times$ 5 rules	Pose
PiPER visual	24 / 6 / 10	1,680 cases $\times$ 2 backends	Episode
Real temporal	4 / 2	10 segments $\times$ 63 conditions	Trajectory
Online PiPER	9 / 27	3 tasks; 240 policy episodes	Session/pair

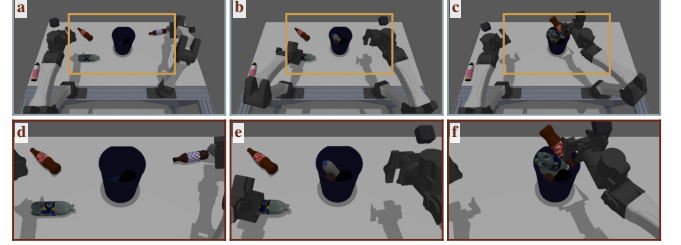


Fig. 4. ABC native-policy execution in the bottles-in-bin scene. Columns correspond to simulation steps 327, 981, and 1635 from seed 42002; the lower row enlarges the gold-boxed interaction region in each overhead view.

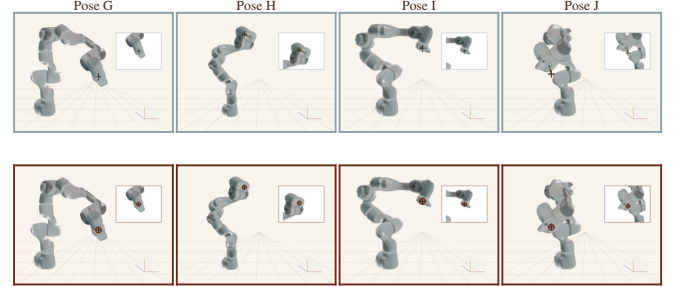


Fig. 5. Controlled Franka recovery on held-out poses G–J. Top: measured state. Bottom: KineSync-GS verified state.

view conflicts; whether the interface transfers across real-world observations and Gaussian backends; whether the same principle improves camera–telemetry alignment under timing disturbances; and whether the complete online pipeline improves manipulation under an unchanged policy. Figure 4 additionally confirms compatibility with the state interface of the public ABC policy and bimanual simulator [22].

a) *Controlled spatial recovery:* We use the seven-joint Franka asset from GS-Playground [3] with two fixed views. Six poses calibrate verification; twelve nonoverlapping poses are held out. Each test pose is rendered under clean or stale-view observations with zero or nonzero measurement offset. All methods receive the same fused inverse estimate, bounded to 0.18 rad and optimized for 60 steps from 480×360 images. We compare measurement retention, unguarded inversion, global verification, component verification, and our whole-state verifier over 48 matched conditions. Figure 5 shows the measured and verified articulated states for four held-out poses.

b) *Real-world visual recovery:* Forty PiPER red-cube episodes are separated into 24 training, six calibration, and ten test episodes. For each test image, Component-GS and Hybrid-GS render a symmetric state window around delayed telemetry. The methods are delayed state, unguarded retrieval, verified retrieval, and a paired-state oracle; all test decisions use thresholds fixed on the calibration split. The

complementary real-robot sequences in Figs. 6 and 7 show the head/wrist observations and task progression represented by the recorded PiPER data.

c) Temporal synchronization: The Take-Pens recordings contain synchronized head/wrist video and six-joint telemetry. Four files provide development data and two held-out trajectories provide ten continuous test segments. We evaluate seven injected offsets under three jitter and three dropout levels, for 630 matched conditions. Raw nearest and linear interpolation, unguarded motion alignment, verified alignment, and an offset oracle share the same perturbed inputs.

d) Online real-robot evaluation: The complete PiPER pipeline consumes synchronized head/external D455 images and six-joint telemetry in one online session. Eighteen held-out 60-second sessions cover three tasks and balanced disturbance segments; nine additional 20-minute sessions test continuous operation. Downstream evaluation holds the policy checkpoint fixed and changes only its state input over 240 paired episodes.

B. Metrics and Statistical Analysis

Joint qMAE averages absolute arm-joint error, reported in degrees for Franka and mrad for PiPER. A harmful spatial update increases qMAE over the input measurement; helpful utilization counts beneficial proposals that are applied. Visual recovery requires qMAE below 10 mrad. Temporal recovery additionally requires offset error below 20 ms. Update precision measures the fraction of applied corrections that improve state, while coverage measures how often a correction is applied when an error is present.

For online operation, end-to-end latency spans image acquisition through verified-state publication. Drift is measured during repeated pose holds, availability counts finite outputs within the state-age budget, and task success requires object placement and two seconds of stable release within 60 seconds. Franka intervals use 10,000 paired pose-cluster bootstrap replicates, with pose-level exact sign-flip tests and Holm correction for the primary comparisons. PiPER visual intervals use 2,000 episode-cluster bootstrap replicates. Temporal metrics are first aggregated within each held-out trajectory and then macro-averaged, keeping the trajectory rather than the perturbation as the independent unit.

V. RESULTS

A. Whole-State Verification Resolves View Conflict

Whole-state verification converts inverse-rendering proposals into reliable state updates. It obtains the lowest aggregate error in Table II, reducing qMAE from 0.590° for the measured state to 0.322° . Unlike the unguarded estimator, it uses visual corrections where the views agree and retains telemetry where their evidence conflicts (Fig. 8). The improvement follows from update selectivity rather than conservative inactivity. KineSync-GS applies 12 beneficial corrections while making no harmful update across all 48 held-out conditions. Figure 9 makes this operating behavior explicit: the method couples high helpful-proposal use with

zero harmful updates, while Table II confirms zero drift from correct inputs.

Optimization traces explain why this separation is necessary. Under stale observations, visual loss continues to fall even as state error rises sharply (Fig. 2). KineSync-GS evaluates cross-view support after fitting, preserving useful inverse-rendering behavior without treating the fitting objective as a state-quality criterion. The same effect appears geometrically: under a stale view, whole-state verification prevents the large distal-link displacement produced by unguarded fitting (Fig. 10), protecting both the joint vector and the Gaussian embodiment it drives.

B. Verified Recovery Transfers to Real-World Data

The proposal–verification interface transfers across visual backends without changing its state contract. Starting from 84.06 mrad delayed-state error, verified retrieval reaches 2.26 mrad with Component-GS and remains below 3 mrad with Hybrid-GS. Accepted states align the rendered gripper with the recorded image evidence.

Verification also improves the quality of the updates that reach the twin. Across both Gaussian backends, beneficial-update precision exceeds 94% while zero-offset inputs are retained more reliably than with unguarded retrieval. The same verification principle improves temporal synchronization across every tested jitter/dropout combination, reaching 97.97% beneficial-update precision and shifting the full 630-condition error distribution downward relative to linear interpolation (Fig. 8b).

C. Cross-View Evidence Sharpens Update Decisions

Factorized calibration sharpens cross-view evidence by separating shared joint-zero error from camera-specific residuals. On paired camera perturbations, dual-view recovery increases from 42.86% to 80.95% (Table III), showing that factorized calibration yields substantially stronger verification evidence. Component re-verification further improves candidate retention, preserving 16 of 18 successful proposals versus 11 for magnitude gating at matched precision. At the image level, the same pattern is visible as observed state, rendered proposal, residual evidence, and verified commit.

The synchronized state is exposed through a robot-independent state interface. Six PiPER joints and a normalized gripper value map consistently to the physical and Gaussian backends, and an OpenVLA-OFT endpoint [21] consumes the resulting state to produce action chunks. The policy therefore receives a single synchronized state representation while remaining decoupled from the synchronization logic.

D. Online Synchronization Improves Real Manipulation

The complete online chain preserves the same proposal–evidence–update contract on newly arriving camera and telemetry streams. Across all three tasks, KineSync-GS remains close to unguarded fusion in recovery while reducing harmful updates by more than an order of magnitude (Table III). The matched module ablation shows that temporal

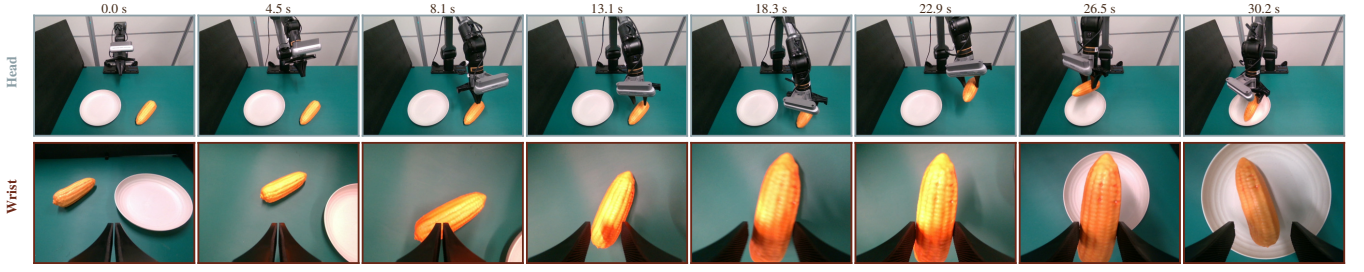
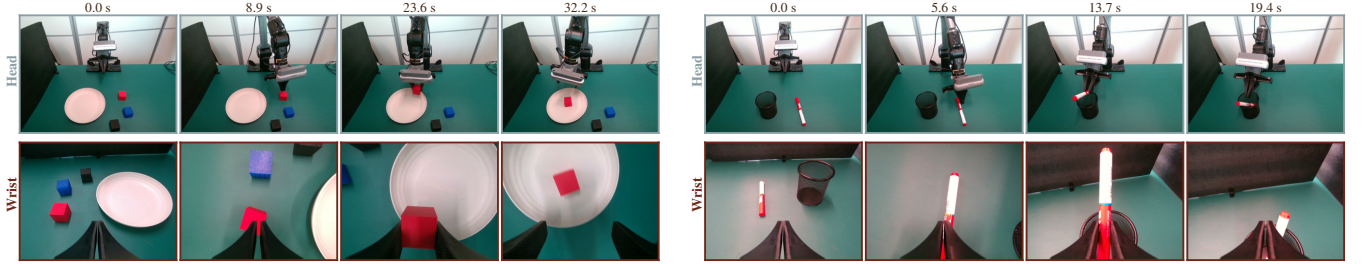


Fig. 6. Real-robot corn-to-plate experiment across matched head and wrist views. Eight temporal columns span the recorded PiPER execution from approach through placement.



(a) Red cube to plate (b) Red pen to bucket
Fig. 7. Real-robot PiPER experiments. (a) Red cube to plate and (b) red pen to bucket at four matched head/wrist times.

TABLE II

WHOLE-STATE RECOVERY ON 12 HELD-OUT FRANKA POSES. QMAE/DRIFT ARE IN DEGREES; BRACKETS ARE 95% POSE-CLUSTER BOOTSTRAP CIs.

Update rule	All qMAE [95% CI] ↓	Clean offset ↓	Stale offset ↓	Stale zero ↓	Harmful /48 ↓	Updates /48	Helpful /13 ↑	Zero drift ↓
Measured state	0.590 [0.566, 0.610]	1.180	1.180	0.000	0	0	0	0.000
Unguarded inverse	2.601 [2.100, 3.018]	0.106	5.073	5.224	23	36	13	2.612
Global verification	1.007 [0.683, 1.325]	0.106	3.921	0.000	8	21	13	0.000
Component verification	0.487 [0.418, 0.561]	0.465	1.482	0.000	6	20	12	0.000
Whole-state verification	0.322 [0.305, 0.337]	0.106	1.180	0.000	0	12	12	0.000

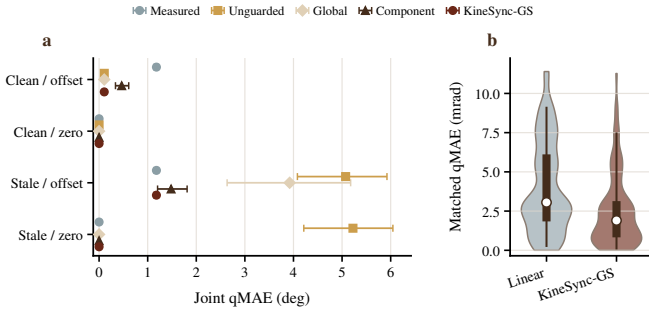


Fig. 8. Spatial and temporal recovery. (a) Franka joint qMAE with 95% pose-cluster bootstrap intervals across four view/offset conditions. (b) PiPER qMAE over 630 matched telemetry conditions; medians decrease from 3.05 to 1.90 mrad.

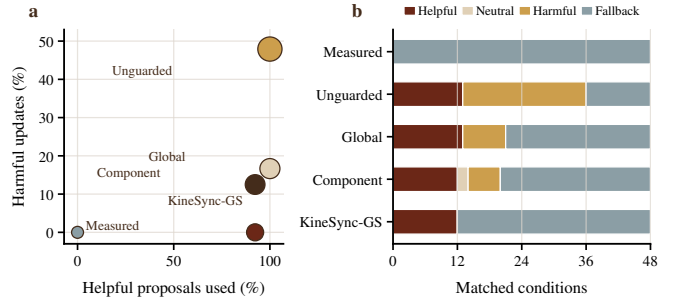


Fig. 9. Verified update behavior on the same 48 held-out conditions. (a) Helpful-proposal utilization versus harmful update rate; marker area encodes applied updates. (b) Update and fallback composition.



Fig. 12. Saved-state spatial verification. Image evidence evaluates the Gaussian proposal and residual before whole-state commitment.

and Gaussian evidence are complementary: their verified combination reaches 3.07 mrad qMAE with only 0.36% harmful updates (Table III). Most importantly, the synchronized state improves manipulation under an unchanged policy, raising task-macro success from 58.33% to 71.67% over 240 paired real-robot episodes. All nine 20-minute runs complete without restart, confirming stable continuous operation (Table III).

TABLE III

ONLINE PIPER SYNCHRONIZATION, SAME-POLICY MANIPULATION, CONTINUOUS OPERATION, AND VERIFIED-UPDATE MECHANISMS. QMAE AND DRIFT ARE IN MRAD AND MRAD/MIN; RECOVERY, HARMFUL UPDATES, FALLBACK, STABILITY, AND AVAILABILITY ARE PERCENTAGES.

Online synchronization over six held-out sessions per task						
Task	Online output	qMAE ↓	Recovery ↑	Harmful ↓	Fallback	
Take-Pens	Raw state	12.84	38.26	0.00	—	
	Temporal only	6.73	62.18	0.83	41.06	
	Combined, ungarded	3.84	81.34	7.48	2.76	
	KineSync-GS	3.46	79.62	0.42	22.36	
Red-Cube-to-Plate	Raw state	11.26	43.71	0.00	—	
	Temporal only	5.88	67.84	0.76	30.27	
	Combined, ungarded	3.28	85.21	6.62	2.14	
	KineSync-GS	2.94	84.36	0.36	19.84	
Corn-to-Plate	Raw state	10.92	46.18	0.00	—	
	Temporal only	5.64	69.12	0.69	26.68	
	Combined, ungarded	3.06	86.44	5.94	2.11	
	KineSync-GS	2.81	85.18	0.31	18.62	
Matched module ablations, macro-averaged over the three tasks						
Variant	Temporal	Gaussian	Verification	qMAE ↓	Recovery ↑	Harmful ↓
Raw state	No	No	None	11.67	42.72	0.00
Temporal only	Yes	No	Temporal	6.08	66.38	0.76
Spatial only	No	Yes	Whole-state	6.21	64.17	0.91
Combined, ungarded	Yes	Yes	None	3.39	84.33	6.68
Component update	Yes	Yes	Component	3.82	76.48	1.37
KineSync-GS	Yes	Yes	Whole-state	3.07	83.05	0.36
Same-policy task success: 40 paired episodes per task and state input						
State input	Take-Pens SR (count)	Red-Cube SR (count)	Corn SR (count)	Task-macro SR (%)	Success time (s) ↓	
Raw state	45.00 (18/40)	62.50 (25/40)	67.50 (27/40)	58.33	16.42	
KineSync state	62.50 (25/40)	75.00 (30/40)	77.50 (31/40)	71.67	14.86	
Three uninterrupted 20-minute sessions per task						
Task	E2E p50/p95 (ms)	Fresh corrections/s	Drift (mrad/min) ↓	Stable/available (%)	Fallback (%)	Completed
Take-Pens	486.73/791.46	1.36	0.18	98.64/98.72	24.38	3/3
Red-Cube-to-Plate	421.58/704.32	1.48	0.12	99.16/99.08	21.47	3/3
Corn-to-Plate	453.26/748.91	1.43	0.15	98.92/98.91	22.81	3/3
Cross-view calibration factorization						
Calibration	Cases	Recovered (count)	Recovery	Gain (pp)	Dual-view	Win/loss
Unfactorized	45	23/45	51.11	—	42.86	—
Factorized	45	29/45	64.44	+13.33	80.95	9/1
Component-level re-verification on matched candidates						
Update rule	Cases	Successful candidates	Retained (count)	Retention	Precision	Coverage
Magnitude gate	24	18	11/18	61.11	100.00	45.83
Component re-verification	24	18	16/18	88.89	100.00	66.67

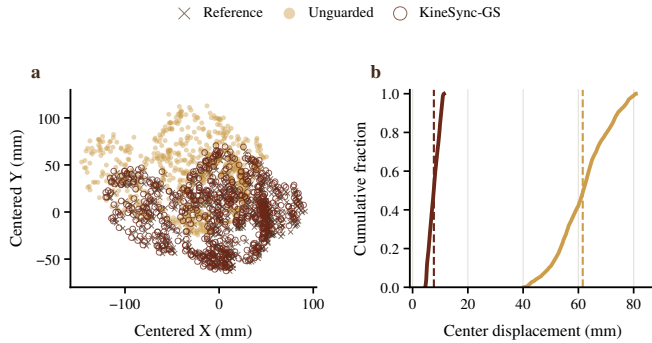


Fig. 10. Distal-link geometry under a stale view. (a) XY projections of corresponding Gaussian centers. (b) Empirical displacement distributions; dashed lines mark means.

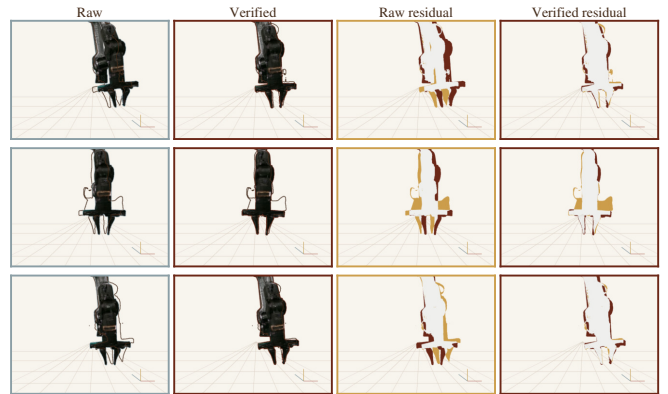


Fig. 11. Real-world PiPER alignment on three matched cases. Columns show measured and verified gripper regions and residuals; gold/brown indicate missed/excess foreground.

VI. CONCLUSION

We introduced KineSync-GS, a verified synchronization framework for articulated Gaussian twins. Its central contribution is to separate state estimation from state-changing authority: rendering or motion proposes a correction, evidence evaluates that proposal, and the twin receives either a verified whole-state update or the preserved measurement. This common interface unifies spatial recovery from multi-view Gaussian observations with temporal camera-telemetry alignment. Controlled Franka experiments show that KineSync-GS preserves useful inverse-rendering corrections while rejecting view-conflicted updates; real-world PiPER experiments show the same behavior across Gaussian backends and timing disturbances. Online evaluation further demonstrates that the synchronized state improves success under an unchanged policy while supporting sustained continuous operation. KineSync-GS therefore turns visual evidence from a rendering signal into a state-changing interface that connects real observations, articulated Gaussian twins, and downstream robot policies.

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.
- [2] S. Yang, *et al.*, “Novel demonstration generation with Gaussian Splatting enables robust one-shot manipulation,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [3] Y. Jia, *et al.*, “GS-Playground: A high-throughput photorealistic simulator for vision-informed robot learning,” in *Proceedings of Robotics: Science and Systems*, 2026.
- [4] X. Li, *et al.*, “RoboGSim: A Real2Sim2Real robotic Gaussian Splatting simulator,” *arXiv preprint arXiv:2411.11839*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.11839>
- [5] X. Han, *et al.*, “RE³SIM: Generating high-fidelity simulation data via 3D-photorealistic real-to-sim for robotic manipulation,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [6] G. Jiang, *et al.*, “GSWorld: Closed-loop photo-realistic simulation suite for robotic manipulation,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [7] K. Zhang, *et al.*, “Real-to-sim robot policy evaluation with gaussian splatting simulation of soft-body interactions,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [8] S. Jain and H. J. Choi, “SplatCtrl: Perception–action coupling via gaussian scene representations and reactive robot control,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [9] T. Schmidt, R. Newcombe, and D. Fox, “DART: Dense articulated real-time tracking,” in *Proceedings of Robotics: Science and Systems*, 2014.
- [10] T. E. Lee, *et al.*, “Camera-to-robot pose estimation from a single image,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [11] J. Lu, F. Richter, and M. C. Yip, “Markerless camera-to-robot pose estimation via self-supervised sim-to-real transfer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 21 296–21 306.
- [12] Y. Labbé, J. Carpentier, M. Aubry, and J. Sivic, “Single-view robot pose and joint angle estimation via render & compare,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 1654–1663.
- [13] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “FoundationPose: Unified 6D pose estimation and tracking of novel objects,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 17 868–17 879.
- [14] R. Liu, A. Canberk, S. Song, and C. Vondrick, “Differentiable robot rendering,” in *Proceedings of the 8th Conference on Robot Learning*, ser. *Proceedings of Machine Learning Research*, vol. 270. PMLR, 2025, pp. 117–129.
- [15] Y. Cai, P. Janssonie, C. de Farias, O. Arenz, and J. Peters, “GaussTwin: Unified simulation and correction with gaussian splatting for robotic digital twins,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [16] S. Lin, *et al.*, “SplArt: Articulation estimation and part-level reconstruction with 3D Gaussian Splatting,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 8841–8851.
- [17] Y. Wang, C. Qian, R. Fan, and E. Johns, “Observer-actor: Active vision imitation learning with sparse-view gaussian splatting,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [18] R. Li and Y.-m. Cheung, “Active learning of 3D gaussian splatting with consistent region partition and robust pose estimation,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [19] P. Furgale, J. Rehder, and R. Siegwart, “Unified temporal and spatial calibration for multi-sensor systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013, pp. 1280–1286.
- [20] J. Kelly, N. Roy, and G. S. Sukhatme, “Determining the time delay between inertial and visual sensor measurements,” *IEEE Transactions on Robotics*, vol. 30, no. 6, pp. 1514–1523, 2014.
- [21] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [22] A. Allshire, *et al.*, “Scalable behavior cloning with open data, training, and evaluation,” *arXiv:2606.27375*, 2026.